

Logical Planning Laboratory

Logic + Search = Planning

This report follows the seven submission items in the lab: planning specification, manual plan, LLM prompt, Python planner, test results, Think About It answers, and LLM reflection.

1. Planning problem specification

Initial state: {At(Robot,A), At(Package,A)}

Goal: {At(Package,C)}

Actions: Move between connected locations, PickUp when the robot and package are at the same place, and Drop when the robot is holding the package at its current location.

For Move(A,B), the precondition is At(Robot,A), with effects -At(Robot,A) and +At(Robot,B). For PickUp(Package,A), the preconditions are At(Robot,A) and At(Package,A), and the effects remove At(Package,A) and add Holding(Package). For Drop(Package,C), the preconditions are At(Robot,C) and Holding(Package), and the effects remove Holding(Package) and add At(Package,C).

Initially PickUp(Package,A) is applicable because both the robot and package are at A. Drop(Package,C) is not applicable because neither At(Robot,C) nor Holding(Package) is true initially.

2. Manual plan

State	Facts
S0	At(Robot,A), At(Package,A)
S1	At(Robot,B), At(Package,A)
S2	At(Robot,A), At(Package,A)
S3	At(Robot,A), Holding(Package)
S4	At(Robot,B), Holding(Package)
S5	At(Robot,C), Holding(Package)
S6	At(Robot,C), At(Package,C)

Plan: Move(A,B) -> Move(B,A) -> PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C).

The PDF gives Move(A,B), PickUp(Package,B), Move(B,C), Drop(Package,C) as an example of actions to reason about, but with the stated initial state the package is at A. I therefore followed the action preconditions literally and used the valid six-step plan above.

3. LLM prompt

The exact prompt is stored in **01_llm_prompt.txt**. It specifies set-based states, positive/negative preconditions and effects, BFS, goal checking, no-plan handling, and printing the state after every action.

4. Generated Python planner

The final implementation is **planner.py**. It checks action applicability using the positive and negative preconditions, applies effects to create successor states, and uses BFS to search for a goal state. The program also reconstructs and prints the action sequence and all intermediate states.

5. Test results

Test	Plan found?	Result
A - Original solvable problem	Yes	Valid plan: PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)
B - Impossible problem (PickUp removed)	No	No plan found
C - Irrelevant robot movement actions	No	No plan found

Test A produced a valid plan that reaches $At(Package, C)$. Test B removes $PickUp$ actions, so no valid plan exists and the program correctly reports failure. Test C contains robot movement actions only; the planner correctly avoids treating $At(Robot, C)$ as equivalent to $At(Package, C)$.

6. Logic and search / Think About It

The logical part checks $S \models \text{Preconditions}(a)$ and determines how the state changes after an applicable action. The search part explores the possible sequences of those actions. The flow is: current state $\rightarrow$ check preconditions $\rightarrow$ choose an applicable action $\rightarrow$ generate successor state $\rightarrow$ search alternatives $\rightarrow$ check the goal. In simple terms, logic decides what is possible, while search decides what to try.

For the optional verification question, the independently executed state transitions should be trusted more than an LLM explanation because they are produced directly by the program and can be checked step by step.

7. Reflection

Why is it useful to specify preconditions and effects first?

It gives the planner an exact description of when an action can happen and how the world changes. That makes the implementation much less ambiguous.

Give an error if preconditions were not checked.

The program might drop the package at C even when the robot is somewhere else or when it is not holding the package, creating an invalid state.

Why isn't a reasonable-looking plan necessarily valid?

Every action has to be applicable in the actual state produced by the previous actions. A list can look reasonable but still contain an impossible step.

What did the LLM contribute?

It helped turn the planning specification into Python data structures, action handling and BFS code.

What did I verify independently?

I verified the preconditions, effects, state transitions, goal test, returned plan, and no-plan cases.

Where is logical reasoning used?

It is used in checking whether preconditions hold and in determining which facts are added or removed after an action.

How is planning related to search?

Planning can be viewed as search over states. Only logically applicable actions create successors, and BFS searches these alternatives until the goal is reached.

8. Optional Prolog extension

The final three pages of the PDF mark the Prolog section as optional. I included the requested **optional_planner.pl** and **optional_prolog_answers.txt** separately in the ZIP. They cover the `connected(a,b)/connected(b,c)` rules, the `valid_move` checks, and the `wet_road -> slippery -> reduce_speed` inference chain.

9. LLM reflection

I used the LLM to help write the first version of the planner, mainly the action representation and BFS structure. I did not rely on its explanation alone. I checked the generated plan by executing the same preconditions and effects in Python and used separate tests for a solvable problem, an impossible problem, and a case where the robot can move without moving the package. That made it easier to catch mistakes that would not be obvious from the generated code alone.